Inteligentna Analiza Danych

Zadania 1–4: przewodnik dla osoby, która ich nie zna

Jędrzej Małaczyński Jędrzej Kostyk

2 listopada 2026

Ten dokument tłumaczy od podstaw, **o co chodzi w zadaniach 1–4**, co dokładnie zrobiliśmy, jak czytać tabele i wykresy ze sprawozdań oraz dlaczego wyniki wyszły tak, a nie inaczej. Zakłada tylko znajomość podstaw algebry (wektor, układ równań). Wszystkie pozostałe pojęcia są objaśnione w rozdziale 1, a na końcu są pytania kontrolne z odpowiedziami i słowniczek.

Spis treści

Jak zadania łączą się ze sobą **2**

1 Podstawy potrzebne do wszystkich zadań **3**

1.1 Wektor, iloczyn skalarny, długość . . . 3
1.2 Neuron liniowy . . . 4
1.3 Zbiór treningowy i błąd . . . 4
1.4 Trening: schodzenie po zboczu . . . 4
1.5 Funkcja sigmoidalna (potrzebna w zadaniu 3) . . . 5
1.6 Ziarno generatora losowego . . . 5

2 Zadanie 1: trening neuronu liniowego regułą delta **6**

2.1 Kluczowa idea: trening to rozwiązywanie układu równań . . . 6
2.2 Reguła delta . . . 6
2.3 Jak przeprowadziliśmy eksperymenty . . . 7
2.4 Przypadek $N < M$: układ sprzeczny . . . 7
2.5 Przypadek $N = M$: jedno rozwiązanie . . . 9
2.6 Przypadek $N > M$: nieskończenie wiele rozwiązań . . . 9
2.7 Obraz geometryczny . . . 9
2.8 Wnioski z zadania 1 . . . 10

3 Zadanie 2: sieć MADALINE do rozpoznawania liter **11**

3.1 Kluczowa idea . . . 11
3.2 Przykład z wykładu: litery 4×4 . . . 11
3.3 Program `font_generator.py` . . . 12

3.4 Program `madaline_ocr.py` . . . 12
3.5 Współczynnik pewności (*confidence*) . . . 13
3.6 Eksperymenty i wyniki . . . 13
3.7 Wnioski z zadania 2 . . . 14

4 Zadanie 3: perceptron wielowarstwowy 4–2–4 **15**
4.1 Cel: kopiowanie przez wąskie gardło . . . 15
4.2 Jak sieć liczy odpowiedź . . . 15
4.3 Propagacja wsteczna (Algorytm 2.1) w słowach . . . 15
4.4 Wynik z biasem: jak czytać Tabelę 1 . . . 16
4.5 Granica decyzyjna . . . 16
4.6 Bias dokładnie . . . 17
4.7 Bez biasu: dlaczego się nie da . . . 18
4.8 Wpływ współczynnika uczenia . . . 18
4.9 Odpowiedzi na pytania z polecenia . . . 18

5 Zadanie 4: sieć SOM do kompresji obrazów **19**
5.1 Co jest tu nowego: uczenie bez nauczyciela . . . 19
5.2 Jak uczy się sieć SOM (Algorytm 2.1) . . . 19
5.3 Pomysł kompresji: słownik typowych ramek . . . 19
5.4 Jak mierzymy wynik: CR i PSNR . . . 20
5.5 Program i eksperymenty . . . 20
5.6 Najważniejsza obserwacja: neurony martwe . . . 21
5.7 Wnioski z zadania 4 . . . 23

6 Pytania kontrolne z odpowiedziami **24**

7 Słowniczek **25**

Dodatek: pliki i uruchamianie **27**

Jak zadania łączą się ze sobą

Pierwsze trzy zadania tworzą drabinkę: od jednego neuronu, przez jedną warstwę neuronów, do sieci z dwiema warstwami. Czwarte wraca do jednej warstwy, ale uczonej *bez nauczyciela*. Każde kolejne zadanie korzysta z pojęć z poprzednich.

	Model	Skąd się biorą wagi	Główne pytanie i odpowiedź
Zad. 1	jeden neuron liniowy	trening regułą delta	Kiedy da się nauczyć neuron bezbłędnie? Zależy od liczby wag N i wzorców M, bo trening to rozwiązywanie układu równań.
Zad. 2	sieć MADALINE (warstwa neuronów liniowych)	**bez treningu**: wagi to po prostu obrazy liter	Czy porównywanie obrazów iloczynem skalarnym rozpozna zaszumione litery? Tak, do ok. 60% szumu.
Zad. 3	perceptron wielowarstwowy 4–2–4	trening propagacją wsteczną	Czy sieć nauczy się kopiować wejście przez wąskie gardło i po co jest bias? Z biasem tak, bez biasu jest to niemożliwe.
Zad. 4	sieć SOM (warstwa neuronów liniowych)	trening **bez nauczyciela** („zwycięzca bierze wszystko”)	Jak dobrze SOM skompresuje obraz? CR 2,6–4,6 przy PSNR 22–33 dB, a więcej neuronów to lepsza jakość i słabsza kompresja.

Każdy rozdział o zadaniu ma ten sam układ: *polecenie w skrócie* → *kluczowa idea* → *co zrobiliśmy* → *wyniki i jak je czytać* → *wnioski.*

1 Podstawy potrzebne do wszystkich zadań

1.1 Wektor, iloczyn skalarny, długość

Wektor to uporządkowana lista liczb, np. $\mathbf{x} = [2, 3]$ albo obraz 128×128 pikseli zapisany jako lista 16 384 zer i jedynek.

Iloczyn skalarny dwóch wektorów tej samej długości to suma iloczynów odpowiadających sobie elementów:

$$\mathbf{w} \cdot \mathbf{x} = w_1 x_1 + w_2 x_2 + \cdots + w_N x_N.$$

Wynik jest jedną liczbą. Przykład: $[0{,}5, -1] \cdot [2, 3] = 0{,}5 \cdot 2 + (-1) \cdot 3 = -2$.

Długość (norma) wektora: $\|\mathbf{x}\| = \sqrt{x_1^2 + \cdots + x_N^2}$. **Normalizacja** to podzielenie wektora przez jego długość, $\mathbf{x}/\|\mathbf{x}\|$, co daje wektor o długości 1 i tym samym kierunku.

Dla wektorów o długości 1 iloczyn skalarny jest równy **kosinusowi kąta** między nimi:

$$\mathbf{w} \cdot \mathbf{x} = \|\mathbf{w}\|\,\|\mathbf{x}\| \cos\alpha = \cos\alpha.$$

Iloczyn skalarny jako miara podobieństwa

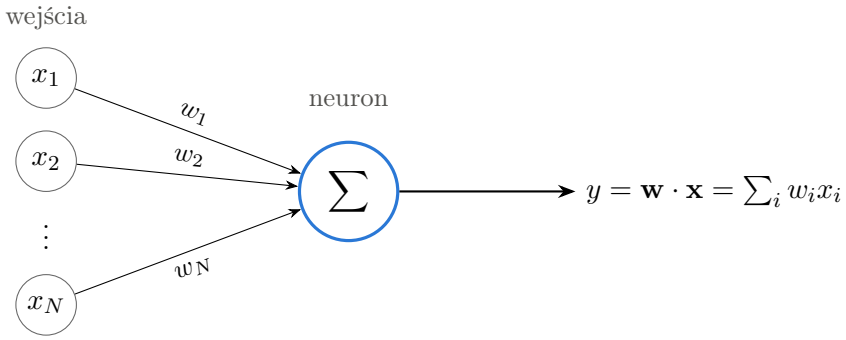
Jeśli wektory mają długość 1 i nieujemne współrzędne (np. obrazy czarno-białe), iloczyn skalarny leży w przedziale $[0, 1]$: 1 oznacza wektory identyczne, 0 wektory „całkowicie różne” (prostopadłe, czyli bez wspólnych niezerowych pozycji). Im większy iloczyn, tym bardziej wektory są do siebie podobne. Na tym opiera się całe zadanie 2.

W Pythonie (NumPy) iloczyn skalarny zapisuje się operatorem `@`:

```
w = np.array([0.5, -1.0]); x = np.array([2.0, 3.0])
w @ x # iloczyn skalarny -> -2.0 (jedna liczba)
w * x # UWAGA: mnozenie element po elemencie -> [1.0, -3.0]
X @ w # macierz (M wierszy) @ wektor -> M iloczynow skalarnych naraz
```

Jeśli wiersze macierzy `X` to kolejne wzorce, to `X @ w` daje odpowiedzi neuronu na wszystkie wzorce jednocześnie.

1.2 Neuron liniowy



Neuron liniowy mnoży każde wejście przez jego **wagę**, sumuje wyniki i podaje sumę na wyjście, czyli liczy iloczyn skalarny $y = \mathbf{w} \cdot \mathbf{x}$. Wagi to „pokrętła" neuronu: od nich zależy, jak odpowiada na dane wejście.

1.3 Zbiór treningowy i błąd

Wzorzec treningowy to para (wejście $\mathbf{x}$, wartość pożądana z), czyli „dla tego wejścia chcemy dostać taką odpowiedź". **Zbiór treningowy** to M takich par. Dla wzorca nr μ błąd neuronu wynosi $e^\mu = z^\mu - y^\mu$.

Błąd średniokwadratowy (MSE) mierzy, jak źle neuron radzi sobie na całym zbiorze:

$$\mathrm{MSE} = \frac{1}{M} \sum_{\mu=1}^{M} \left(z^\mu - y^\mu\right)^2.$$

Kwadrat sprawia, że błędy dodatnie i ujemne się nie znoszą, a duże błędy ważą więcej. W zadaniu 3 używamy wariantu $E = \frac{1}{2}\sum(z-y)^2$ (suma zamiast średniej, dla wygody pochodnych), ale idea jest ta sama.

1.4 Trening: schodzenie po zboczu

Trenowanie (uczenie) neuronu to szukanie takich wag, przy których błąd jest najmniejszy. Błąd zależy od wag, więc można o nim myśleć jak o **krajobrazie**: każdemu ustawieniu wag odpowiada „wysokość" równa błędowi, a my szukamy najniższego punktu.

- **Gradient** to kierunek, w którym błąd rośnie najszybciej. Robimy krok w przeciwną stronę, czyli „w dół zbocza".
- **Współczynnik uczenia** η (eta) to długość tego kroku. Za mały daje powolny trening, za duży powoduje „przeskakiwanie" dołka i niestabilność.
- **Epoka** to jedno przejście przez wszystkie wzorce treningowe.
- W wariancie **on-line** (używanym we wszystkich naszych zadaniach) wagi poprawia się po *każdym* wzorcu, a nie raz na epokę.

1.5 Funkcja sigmoidalna (potrzebna w zadaniu 3)

$$f(s) = \frac{1}{1 + e^{-s}}, \qquad f(0) = 0{,}5, \qquad f(s) \to 1 \text{ dla } s \to +\infty, \qquad f(s) \to 0 \text{ dla } s \to -\infty.$$

„Miękki przełącznik": zamienia dowolną liczbę na wartość z przedziału $(0, 1)$. Duża dodatnia suma daje prawie 1, duża ujemna prawie 0, a suma bliska zeru daje „niezdecydowane" 0,5. Jej pochodna ma wygodną postać $f'(s) = f(s)\,(1 - f(s))$.

1.6 Ziarno generatora losowego

Komputer losuje liczby z generatora, który po ustawieniu tego samego **ziarna** (ang. *seed*) zawsze daje ten sam ciąg. Dzięki temu nasze „losowe" dane i wagi są powtarzalne: po ponownym uruchomieniu kodu wychodzą identyczne liczby jak w sprawozdaniach.

2 Zadanie 1: trening neuronu liniowego regułą delta

Polecenie w skrócie

Zaimplementuj trening jednego neuronu liniowego regułą delta (Algorytm 3.2 z Wykładu 1) na wielu wzorcach treningowych i zbadaj trzy przypadki: liczba wag N **mniejsza**, **równa** i **większa** od liczby wzorców M. W każdym przypadku:

- uruchom trening kilka razy z **różnymi losowymi wagami początkowymi**, ale za każdym razem na **tym samym** (losowym) zbiorze treningowym,
- porównaj wagi końcowe z różnych uruchomień,
- sprawdź, czy udało się nauczyć neuron bezbłędnie.

Obserwacje zinterpretuj, wiedząc, że trening neuronu liniowego to rozwiązywanie układu M równań z N niewiadomymi.

2.1 Kluczowa idea: trening to rozwiązywanie układu równań

Chcemy, żeby dla każdego wzorca odpowiedź neuronu była równa wartości pożądanej: $\mathbf{w} \cdot \mathbf{x}^\mu = z^\mu$. To jest **jedno równanie liniowe**, w którym niewiadomymi są wagi. M wzorców daje M równań z N niewiadomymi. Po ułożeniu wzorców jako wierszy macierzy X zapisujemy to krótko jako

$$X\mathbf{w} = \mathbf{z}.$$

Nasz przypadek $N = 2$, $M = 6$: sześć równań, dwie niewiadome

Zbiór treningowy został wylosowany (ziarno 42). Każdy wiersz to jeden wzorec:

$$\begin{aligned} 0{,}548\, w_1 - 0{,}122\, w_2 &= 0{,}288 \\ 0{,}717\, w_1 + 0{,}395\, w_2 &= 0{,}646 \\ -0{,}812\, w_1 + 0{,}951\, w_2 &= -0{,}113 \\ 0{,}522\, w_1 + 0{,}572\, w_2 &= -0{,}546 \\ -0{,}744\, w_1 - 0{,}099\, w_2 &= 0{,}109 \\ -0{,}258\, w_1 + 0{,}854\, w_2 &= -0{,}872 \end{aligned}$$

Liczby nie mają znaczenia fizycznego. Polecenie wymagało losowego zbioru, który jest identyczny w każdym uruchomieniu.

Z algebry wiemy, ile rozwiązań ma taki układ (dla danych losowych):

Przypadek	**Układ**	**Rozwiązań**	**Analogia**
$N < M$ (2 wagi, 6 wzorców)	sprzeczny	0	6 warunków dla 2 pokręteł: za dużo żądań
$N = M$ (4 i 4)	oznaczony	dokładnie 1	tyle warunków, ile pokręteł
$N > M$ (6 wag, 3 wzorce)	nieoznaczony	nieskończenie wiele	za mało warunków, część pokręteł „luzem”

Cała reszta zadania to sprawdzenie, że reguła delta zachowuje się dokładnie tak, jak przewiduje ta tabelka.

2.2 Reguła delta

Po pokazaniu wzorca $\mathbf{x}$ i policzeniu błędu $e = z - y$ wagi poprawia się wzorem

$$\mathbf{w} \leftarrow \mathbf{w} + \eta\, e\, \mathbf{x}.$$

Intuicja

Jeśli odpowiedź była za mała ($e > 0$), przesuwamy wagi w stronę wektora $\mathbf{x}$. Wtedy $\mathbf{w} \cdot \mathbf{x}$ rośnie i odpowiedź zbliża się do celu. Gdy odpowiedź była za duża, przesuwamy je w przeciwną stronę. Matematycznie to krok w dół gradientu błędu $\frac{1}{2}(z - \mathbf{w} \cdot \mathbf{x})^2$ dla jednego wzorca.

Jeden krok reguły delta

$\mathbf{w} = [0{,}5, -1]$, $\mathbf{x} = [2, 3]$, $z = 1$, $\eta = 0{,}05$.

- odpowiedź: $y = 0{,}5 \cdot 2 - 1 \cdot 3 = -2$, błąd $e = 1 - (-2) = 3$,
- nowe wagi: $\mathbf{w} = [0{,}5, -1] + 0{,}05 \cdot 3 \cdot [2, 3] = [0{,}8, -0{,}55]$,
- nowa odpowiedź: $0{,}8 \cdot 2 - 0{,}55 \cdot 3 = -0{,}05$, więc błąd spadł z 3 do 1,05.

Błąd na tym wzorcu maleje o czynnik $1 - \eta\|\mathbf{x}\|^2 = 1 - 0{,}05 \cdot 13 = 0{,}35$. Gdyby $\eta\|\mathbf{x}\|^2 > 2$, krok przeskakiwałby cel i błąd by rósł, dlatego η musi być małe.

Kod treningu (`neuron_liniowy.py`) robi dokładnie to:

```
def delta_training(X, Z, eta=0.01, K=100, w_range=(-1, 1), rng=None):
    M, N = X.shape # M wzorcow, N wag
    w0 = rng.uniform(w_range[0], w_range[1], N) # losowe wagi poczatkowe
    w = w0.copy()
    for k in range(K): # K epok
        for mu in range(M): # kazdy wzorzec po kolei
            y = w @ X[mu] # odpowiedz neuronu
            e = Z[mu] - y # blad
            w = w + eta * e * X[mu] # regula delta
        mse[k] = np.mean((Z - X @ w) ** 2) # blad na calym zbiorze po epoce
    return w0, w, mse
```

2.3 Jak przeprowadziliśmy eksperymenty

- **Zbiór treningowy:** wejścia i wartości pożądane losowane z przedziału $(-1, 1)$, ziarno 42, czyli ten sam zbiór we wszystkich uruchomieniach.
- **Wagi początkowe:** losowane z $(-1, 1)$, inne ziarno w każdym uruchomieniu (100, 101, …, 104), więc 5 różnych startów.
- $\eta = 0{,}01$, $K = 10\,000$ epok.
- **Punkt odniesienia** $X^{+}\mathbf{z}$: „najlepsze możliwe” wagi policzone gotowym wzorem (pseudoodwrotność macierzy, `numpy.linalg.pinv`), bez żadnego treningu. Dla $N < M$ to rozwiązanie metodą najmniejszych kwadratów, dla $N = M$ zwykłe $X^{-1}\mathbf{z}$, a dla $N > M$ rozwiązanie o najmniejszej długości wektora wag.

Dlaczego pierwsze wagi startowe się powtarzają

Uruchomienie nr 1 zaczyna się od 0,6700; 0,1931 zarówno dla $N = 2$, jak i dla $N = 4$ i $N = 6$. To nie błąd: generator z ziarnem 100 zawsze daje ten sam ciąg liczb, a przypadki różnią się tylko tym, ile liczb z niego pobieramy (2, 4 albo 6).

2.4 Przypadek $N < M$: układ sprzeczny

Żadna para (w_1, w_2) nie spełnia wszystkich sześciu równań naraz. Trening nie może więc osiągnąć zerowego błędu, tylko **minimalizuje** MSE, czyli szuka wag, przy których suma kwadratów „niedopasowań” wszystkich równań jest najmniejsza.

Minimum leży tam, gdzie pochodna błędu się zeruje. Daje to tzw. **równania normalne** $X^T X\mathbf{w} = X^T\mathbf{z}$, tu zwykły układ 2×2:

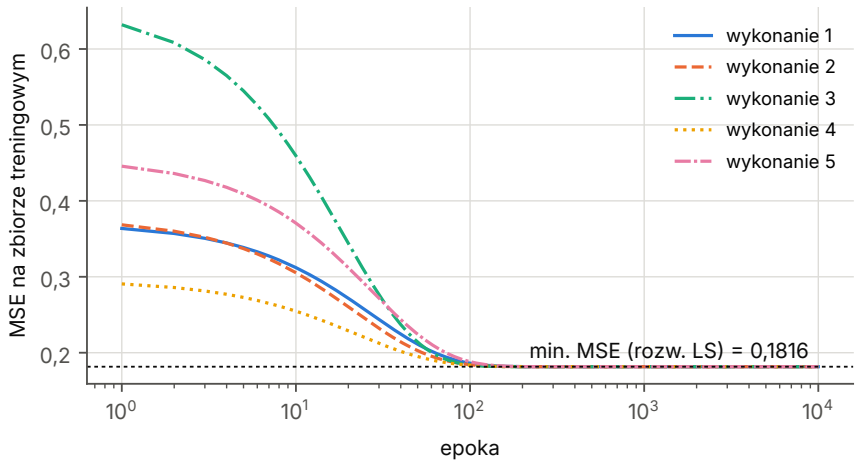
$$\begin{aligned} 2{,}366\, w_1 - 0{,}404\, w_2 &= 0{,}572 \\ -0{,}404\, w_1 + 2{,}141\, w_2 &= -0{,}956 \end{aligned} \qquad \Rightarrow \qquad \mathbf{w} = (0{,}171;\ -0{,}414) = X^+\mathbf{z}.$$

Jak czytać Tabelę 1 ze sprawozdania:

- **wiersze 1–5:** pięć uruchomień treningu, każde z innego startu,
- $w_1^{(0)}, w_2^{(0)}$**:** wagi początkowe (losowe, indeks (0) oznacza krok zerowy),
- w_1, w_2**:** wagi końcowe po 10 000 epokach,
- **MSE:** błąd średniokwadratowy przy wagach końcowych,
- $\max|z - y|$**:** największy błąd na pojedynczym wzorcu, czyli najgorzej dopasowane równanie,
- **ostatni wiersz** X^+z**:** rozwiązanie wzorcowe policzone wzorem. Nie ma wag startowych, bo nie jest wynikiem iteracji.

Co z niej wynika:

- Wagi startowe są wszędzie inne, a końcowe **identyczne** (0,1688; −0,4164, różnica między uruchomieniami 10^{-15}). Krajobraz błędu to „miska” (paraboloida) z jednym dnem, więc każdy start prowadzi w to samo miejsce.
- MSE = 0,1816, a nie 0. Dno miski leży powyżej zera, bo układ jest sprzeczny. Największy błąd na pojedynczym wzorcu to 0,689.
- **Bezbłędny trening nie jest możliwy.**



Rysunek 1: $N = 2$, $M = 6$: błąd w kolejnych epokach. Wszystkie 5 uruchomień schodzi do tego samego, niezerowego minimum.

Dlaczego 0,1688**, a nie dokładnie** 0,1710 **(Tabela 2 ze sprawozdania)?** Wagi są poprawiane po każdym wzorcu, a każdy wzorzec „ciągnie” je w stronę *swojego* równania, a nie wspólnego minimum. To jak sześć osób ciągnących linę po kolei w różne strony: punkt nie stoi idealnie w środku, tylko kręci się wokół niego, i to tym mocniej, im dłuższe są „szarpnięcia” (czyli η). Tabela 2 to potwierdza:

η	odległość od X^+z	nadwyżka MSE ponad minimum	odległość / η
0,1	$3{,}31 \cdot 10^{-2}$	$3{,}36 \cdot 10^{-4}$	0,331
0,01	$3{,}23 \cdot 10^{-3}$	$3{,}19 \cdot 10^{-6}$	0,323
0,001	$3{,}22 \cdot 10^{-4}$	$3{,}18 \cdot 10^{-8}$	0,322

Przy 10 razy mniejszym η odległość maleje 10 razy (ostatnia kolumna jest stała), a nadwyżka błędu 100 razy, bo przy dnie miski błąd rośnie z kwadratem odległości. Dla $\eta \to 0$ wynik dąży dokładnie do rozwiązania najmniejszych kwadratów. Nie jest to więc błąd w kodzie, tylko właściwość wariantu on-line.

2.5 Przypadek $N = M$: jedno rozwiązanie

Cztery równania z czterema niewiadomymi mają dokładnie jedno rozwiązanie $X^{-1}\mathbf{z} = (1{,}856;\ 0{,}746;\ -2{,}142;\ 1{,}823)$.

- Błąd maleje wykładniczo (na wykresie z logarytmiczną osią to prosta) i dąży do zera. **Trening bezbłędny jest możliwy.**
- Po 10 000 epokach wagi są zgodne do trzeciego miejsca po przecinku (MSE $\approx 10^{-6}$), a po 30 000 epokach odległość od $X^{-1}\mathbf{z}$ spada do $5 \cdot 10^{-8}$. Wszystkie starty dochodzą więc do **tego samego** punktu.
- Zbieżność jest wolniejsza niż przy $N < M$, bo macierz X jest gorzej uwarunkowana (cond = 6,79). W jednym kierunku „miska" jest bardzo płaska, więc schodzenie w nim trwa długo.

2.6 Przypadek $N > M$: nieskończenie wiele rozwiązań

Trzy równania z sześcioma niewiadomymi: rozwiązań jest nieskończenie wiele.

- Błąd spada do 10^{-30}, czyli do granicy dokładności liczb w komputerze. **Trening zawsze się udaje.**
- Ale **wagi końcowe są za każdym razem inne** (różnią się nawet o 1,76). Każdy z tych wektorów jest równie dobrym rozwiązaniem.

Dlaczego wynik zależy od startu

Weźmy $N = 3$ i jeden wzorzec $\mathbf{x} = [1, 0, 0]$. Poprawka to zawsze $\eta\, e\, \mathbf{x}$, więc zmienia się tylko w_1. Wagi w_2 i w_3 nie mają żadnego wpływu na odpowiedź, więc trening ich nie rusza: zostają takie, jakie wylosowano. Ogólnie reguła delta poprawia wagi **tylko w kierunkach wzorców**. Część wag „niewidoczna" dla wzorców (tzw. jądro macierzy X) przechodzi z wag startowych bez zmian.

Daje to dokładny wzór na wynik treningu:

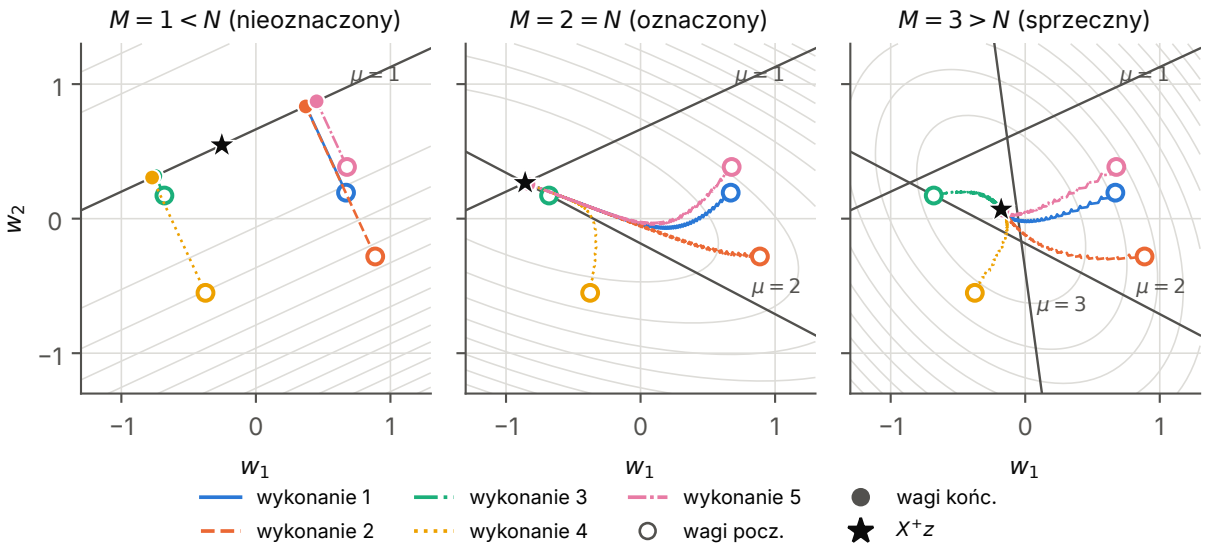
$$\hat{\mathbf{w}} = \underbrace{X^{+}\mathbf{z}}_{\text{rozwiązanie wzorcowe}} + \underbrace{(I - X^{+}X)\,\mathbf{w}_0}_{\text{nieruszona część wag startowych}} .$$

W eksperymencie wagi z treningu zgadzały się z tym wzorem z dokładnością 10^{-14}. Rozwiązanie o najmniejszej długości ($X^{+}\mathbf{z}$) dostalibyśmy tylko startując od wag zerowych.

2.7 Obraz geometryczny

Przy dwóch wagach każde równanie $w_1 x_1 + w_2 x_2 = z$ to **prosta** na płaszczyźnie (w_1, w_2).

- **1 wzorzec** ($M < N$): rozwiązania to cała prosta. Każdy start „spada" prostopadle na prostą, więc końce są różne.
- **2 wzorce** ($M = N$): dwie proste przecinają się w jednym punkcie i wszystkie drogi tam prowadzą.



Rysunek 2: Neuron z 2 wagami: każde równanie (wzorzec) to prosta na płaszczyźnie wag. Kolorowe linie to drogi wag w trakcie treningu z 5 różnych startów (kółka puste: start, pełne: koniec).

- **3 wzorce** ($M > N$): trzy proste nie mają wspólnego punktu. Wszystkie drogi kończą w tym samym miejscu wewnątrz trójkąta (kompromis), z drobnymi zygzakami od poprawek on-line.

2.8 Wnioski z zadania 1

	$N < M$	$N = M$	$N > M$
układ równań	sprzeczny	oznaczony	nieoznaczony
błąd po treningu	0,1816 (minimum)	$\to 0$	$\approx 10^{-30}$
wagi końcowe z różnych startów	identyczne	identyczne	różne
bezbłędny trening	**nie**	**tak**	**tak**

Wszystko wynika z tego, ile rozwiązań ma układ $X\mathbf{w} = \mathbf{z}$. Praktyczny morał: zerowy błąd przy większej liczbie wag niż danych nie oznacza dobrego modelu, bo część wag jest po prostu losowa.

3 Zadanie 2: sieć MADALINE do rozpoznawania liter

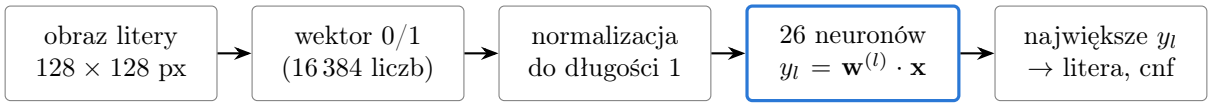
Polecenie w skrócie

Napisz dwa programy uruchamiane z linii poleceń:

- `font_generator.py`: rysuje jedną literę z czcionki `.ttf` na obrazie $w \times h$ w punkcie (x, y), dodaje zadany procent szumu, zapisuje `litera.png` i dopisuje opis do pliku `description.txt`,
- `madaline_ocr.py`: z obrazów wzorcowych buduje sieć MADALINE, rozpoznaje obrazy testowe i wypisuje `test -> rozpoznana litera, confidence: liczba`.

Sprawdź skuteczność dla szumu 10%, 30%, 50%, 90% oraz poprawność dla 0% (wzorce rozpoznawane same przez siebie muszą dać pewność 1).

3.1 Kluczowa idea

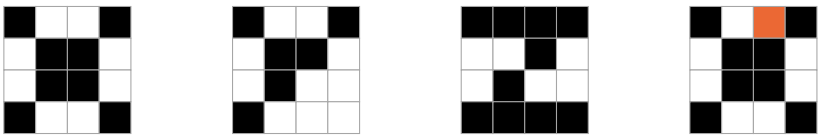


1. Obraz czarno-biały zamieniamy na wektor: czarny piksel to 1, biały to 0, wiersz po wierszu (*wektoryzacja*).
2. Wektor normalizujemy do długości 1, bo wtedy iloczyn skalarny jest miarą podobieństwa (rozdział 1.1).
3. **Sieć MADALINE** (*Multiple ADAptive LINear Elements*) to warstwa neuronów liniowych, z których każdy dostaje ten sam obraz. **Jeden neuron odpowiada jednej literze**, a jego wagi to po prostu znormalizowany obraz tej litery.
4. Każdy neuron liczy więc podobieństwo obrazu testowego do „swojej" litery. Wygrywa neuron z największym wyjściem (*neuron zwycięski*).

Ta sieć się nie uczy

W odróżnieniu od zadań 1 i 3 nie ma tu żadnego treningu iteracyjnego. Wagi ustawia się raz, kopiując wzorce liter. Sieć to w gruncie rzeczy porównywanie z szablonem: „do którego wzorca ten obraz jest najbardziej podobny?".

3.2 Przykład z wykładu: litery 4×4



litera X (wzorzec) litera Y (wzorzec) litera Z (wzorzec) zaburzona X (test)

Pomarańczowy piksel to zaburzenie: piksel dodany w obrazie testowym względem litery X. Wektor litery X to wiersze sklejone jeden za drugim: $[1,0,0,1,\ 0,1,1,0,\ 0,1,1,0,\ 1,0,0,1]$.

Dla obrazów 0/1 iloczyn znormalizowanych wektorów upraszcza się do liczenia pikseli:

$$\text{podobiestwo} = \frac{\text{liczba wspólnych czarnych pikseli}}{\sqrt{n_{\text{wzorzec}} \cdot n_{\text{test}}}}.$$

Rozpoznanie „zaburzonej X”

Obraz testowy ma 9 czarnych pikseli (8 z litery X i jeden dodatkowy).

- X: 8 pikseli, wspólnych 8, więc $8/\sqrt{8 \cdot 9} = \mathbf{0{,}943}$,
- Y: 6 pikseli, wspólnych 6, więc $6/\sqrt{6 \cdot 9} = 0{,}816$,
- Z: 10 pikseli, wspólnych 7, więc $7/\sqrt{10 \cdot 9} = 0{,}738$.

Największe podobieństwo ma X, więc sieć rozpoznaje X z pewnością 0,943. To dokładnie wartości podane w materiałach do wykładu.

Podobieństwo wynosi 1 tylko dla obrazu identycznego ze wzorcem, a 0 dla obrazu, który nie ma z nim ani jednego wspólnego czarnego piksela.

3.3 Program `font_generator.py`

```
python font_generator.py 128 128 28 10 times.ttf a 30 test_patterns_30
# w h x y czcionka litera szum[%] katalog
```

1. Rysuje literę z pliku czcionki w punkcie (x, y) na białym obrazie $w \times h$. Rozmiar czcionki nie jest podany w poleceniu, więc wybieramy największy, przy którym cała litera (z ogonkami jak w „g”) mieści się w obrazie. Dla naszych parametrów wychodzi 102 px.
2. Zamienia obraz na czysto czarno-biały (próg 128).
3. Dodaje szum: **każdy piksel z prawdopodobieństwem** $p/100$ **dostaje losową wartość** (czarny albo biały, po połowie).
4. Zapisuje `a.png` i dopisuje do `description.txt` linię `a.png:letter a, noise level 30%`. Ten plik mówi drugiemu programowi, jakie obrazy są w katalogu i jak je podpisać.

Dlaczego taki model szumu

Materiały nie podają wzoru na szum. Przy naszym modelu 0% to obraz czysty, a 100% to obraz całkowicie losowy. Model odtwarza też przykładowe wartości pewności z wykładu. Alternatywa, czyli *odwrócenie* p% pikseli, przy 90% dałaby negatyw litery (czarne tło, biała litera), a nie zaszumioną literę.

3.4 Program `madaline_ocr.py`

```
python madaline_ocr.py training_patterns test_patterns_30
```

1. Czyta `description.txt` z katalogu wzorców, wczytuje obrazy, zamienia je na wektory i normalizuje. Wektory stają się wierszami macierzy wag W (26 wierszy, jeden na literę).
2. To samo robi z obrazami testowymi.
3. Dla każdego obrazu testowego liczy $\mathbf{y} = W\mathbf{x}$ (wyjścia wszystkich 26 neuronów naraz), wybiera największe i wypisuje wynik:

```
letter a, noise level 30% -> letter a, noise level 0%, confidence: 0.449
```

Lewa strona to etykieta obrazu testowego, prawa to etykieta neuronu zwycięskiego.

3.5 Współczynnik pewności (*confidence*)

To **wyjście neuronu zwycięskiego**, czyli kosinus kąta między znormalizowanym obrazem testowym a wzorcem litery, która wygrała.

Litera „a” przy 30% szumu

Wzorzec „a” ma 825 czarnych pikseli. Obraz testowy ma ich 2975 (litera plus szum). Wspólnych jest 704.

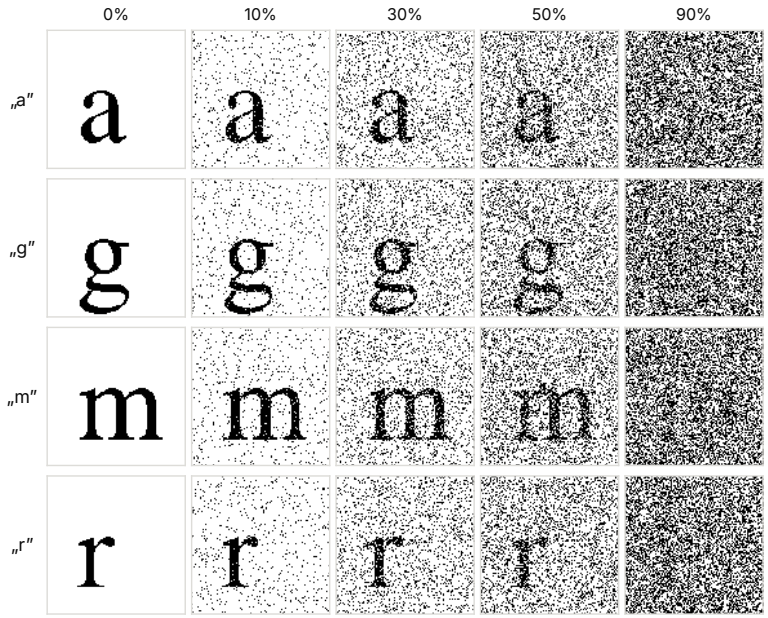
$$cnf = \frac{704}{\sqrt{825 \cdot 2975}} = \frac{704}{1566{,}6} = 0{,}449,$$

dokładnie jak w raporcie `raport_szum_30.txt`.

- Pewność spada przy szumie **nawet wtedy, gdy rozpoznanie jest poprawne**. Szum dokłada mnóstwo czarnych pikseli, więc mianownik mocno rośnie, a licznik (wspólne piksele) trochę maleje.
- To **nie jest prawdopodobieństwo**: 0,449 nie znaczy „45% pewności, że to a”.
- O poprawności decyduje to, czy neuron właściwej litery ma wyższe wyjście niż *wszystkie pozostałe*, a nie bezwzględna wartość pewności.

3.6 Eksperymenty i wyniki

Założenia: 26 małych liter a–z, czcionka Times New Roman, obrazy 128×128, litera w punkcie $(28, 10)$. Sieć ma 26 neuronów, a wzorce to litery bez szumu.



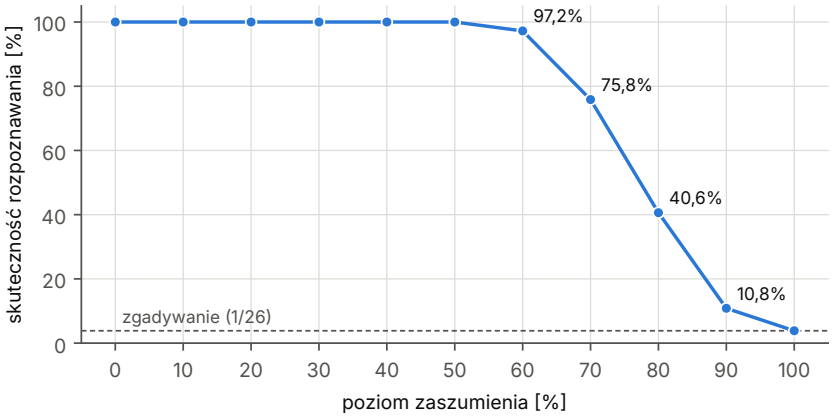
Rysunek 3: Przykładowe obrazy testowe dla poziomów szumu 0–90%.

Eksperyment 1 (wymagany poleceniem). Jeden komplet 26 liter na każdy poziom szumu:

szum	0%	10%	30%	50%	90%
poprawnie rozpoznane	26/26	26/26	26/26	26/26	4/26
średnia pewność	1,000	0,695	0,446	0,325	0,215

Wynik dla 0% (pewność dokładnie 1,000 dla każdej litery) potwierdza, że programy działają poprawnie. Do 50% szumu sieć się nie myli. Przy 90% aż 22 litery zostały rozpoznane jako „m”.

Eksperyment 2 (dokładniej). Pojedyncze losowanie to za mało, więc dla poziomów 0–100% powtórzyliśmy szum 50 razy dla każdej litery (1300 rozpoznań na poziom).



Rysunek 4: Skuteczność w funkcji szumu: 100% do 50% szumu, potem szybki spadek do poziomu zgadywania (1/26).

Dlaczego przy dużym szumie wygrywa „m”

„m” ma najwięcej czarnych pikseli (1462, a „i” tylko 535). Szum rozsiewa czarne piksele równomiernie po całym obrazie, a duży wzorzec „łapie” ich najwięcej. Gdy prawdziwy kształt litery ginie w szumie, wygrywa po prostu największy wzorzec. Dla obrazu całkowicie losowego „m” wygrało we wszystkich 2000 próbach. Pierwsze błędy (od 60%) dotyczą liter cienkich, np. r→m, i→m.

Eksperyment 3: przesunięcie i inny krój. Litery bez szumu, ale przesunięte albo w innej czcionce:

wariant	skuteczność	wariant	skuteczność
przesunięcie o 1 px	100%	Liberation Serif (krój prawie jak Times)	100%
przesunięcie o 2 px	96%	DejaVu Serif (inne proporcje)	23%
przesunięcie o 4 px	42%	Liberation Sans (bez szeryfów)	38%
przesunięcie o 8 px	4%		

Sieć porównuje obrazy piksel po pikselu, więc przesunięcie o 4 piksele szkodzi bardziej niż 70% szumu. Rozpoznaje dobrze tylko litery narysowane tym samym krojem i w tym samym miejscu co wzorce.

3.7 Wnioski z zadania 2

- Iloczyn skalarny znormalizowanych obrazów to prosta i skuteczna miara podobieństwa. Sieć jest bardzo odporna na losowy szum (100% skuteczności do 50%).
- Współczynnik pewności to kosinus kąta, a nie procent pewności. Maleje szybko nawet przy poprawnym rozpoznaniu.
- Przy dużym szumie sieć myli się systematycznie na korzyść „ciężkich” liter.
- Słabość: brak odporności na przesunięcie i zmianę kroju. Pomogłoby wyrównanie położenia i rozmiaru znaku przed rozpoznaniem.

4 Zadanie 3: perceptron wielowarstwowy 4–2–4

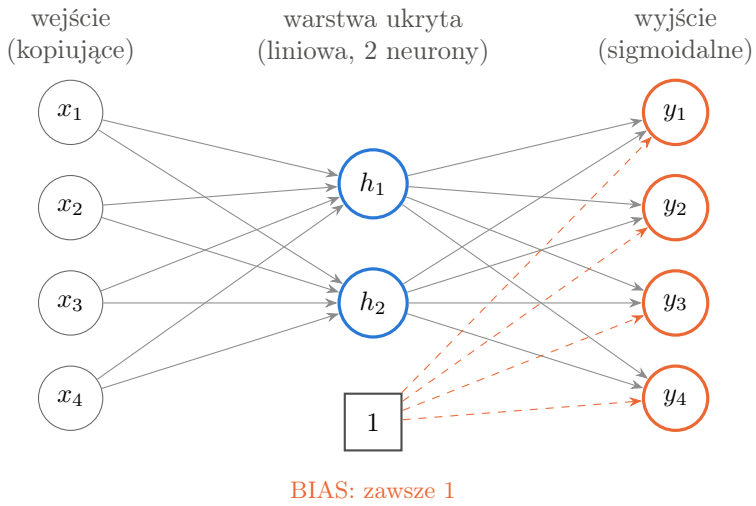
Polecenie w skrócie

Zaimplementuj trening sieci typu perceptron wielowarstwowy metodą wstecznej propagacji błędu (Algorytm 2.1 z Wykładu 2). Architektura: 4 neurony wejściowe (kopiujące), 2 neurony liniowe w warstwie ukrytej, 4 neurony sigmoidalne z wejściem **BIAS** w warstwie wyjściowej. Zbiór treningowy to 4 wzorce, w których wyjście ma być równe wejściu:

$$[1,0,0,0] \to [1,0,0,0], \quad [0,1,0,0] \to [0,1,0,0], \quad [0,0,1,0] \to [0,0,1,0], \quad [0,0,0,1] \to [0,0,0,1].$$

Odpowiedz: czy sieć dałoby się wytrenować **bez biasu** i jak intuicyjnie rozumieć rolę biasu?

4.1 Cel: kopiowanie przez wąskie gardło



Sieć ma odtworzyć na wyjściu to, co dostała na wejściu. Samo kopiowanie byłoby trywialne, ale informacja musi przejść przez **wąskie gardło**: tylko 2 neurony ukryte. Sieć musi więc sama wymyślić **kod**: zapisać każdy z 4 wzorców jako parę liczb (h_1, h_2), a potem z tej pary go odtworzyć. Nikt jej tego kodu nie podaje. Taką sieć nazywa się *koderem* (autoenkoderem).

Zadanie służy do sprawdzenia implementacji propagacji wstecznej na prostym przykładzie oraz do pokazania, że bez biasu takiego kodu nie da się zbudować.

4.2 Jak sieć liczy odpowiedź

$$h_i = \sum_{j=1}^{4} w_{ij}^{(1)} x_j \quad (i = 1, 2), \qquad s_i = w_{i1}^{(2)} h_1 + w_{i2}^{(2)} h_2 + w_{i3}^{(2)} \cdot 1, \qquad y_i = f(s_i) \quad (i = 1, \ldots, 4).$$

Indeks (1) oznacza wagi warstwy ukrytej, (2) warstwy wyjściowej. Waga $w_{i3}^{(2)}$ to **waga biasu** neuronu i (mnożona przez stałe wejście 1). Sieć ma $8 + 12 = 20$ wag.

Wejścia mają tylko jedną jedynkę, więc kod wzorca nr k w warstwie ukrytej to po prostu k-ta kolumna macierzy wag $W^{(1)}$, czyli **jeden punkt na płaszczyźnie** (h_1, h_2).

4.3 Propagacja wsteczna (Algorytm 2.1) w słowach

Dla każdego wzorca w każdej epoce:

1. **W przód:** policz h_1, h_2, potem s_i i wyjścia $y_i = f(s_i)$.
2. **Błąd wyjścia:** $\delta_i^{(2)} = f'(s_i)\,(z_i - y_i)$, czyli błąd pomnożony przez „czułość” sigmoidy w tym punkcie.
3. **Błąd warstwy ukrytej (wstecz):** $\delta_i^{(1)} = \sum_k w_{ki}^{(2)}\,\delta_k^{(2)}$. Neuron ukryty dostaje „część winy” za każdy błąd wyjścia, proporcjonalną do tego, jak mocno na to wyjście wpływał (waga połączenia). Stąd nazwa: **błędy płyną od wyjścia wstecz**.
4. **Korekta wag:** $w_{ij}^{(1)} \mathrel{+}= \eta\,\delta_i^{(1)}\,x_j$ oraz $w_{ij}^{(2)} \mathrel{+}= \eta\,\delta_i^{(2)}\,h_j$. To ta sama zasada co w regule delta: błąd neuronu × sygnał wchodzący do wagi × krok η.

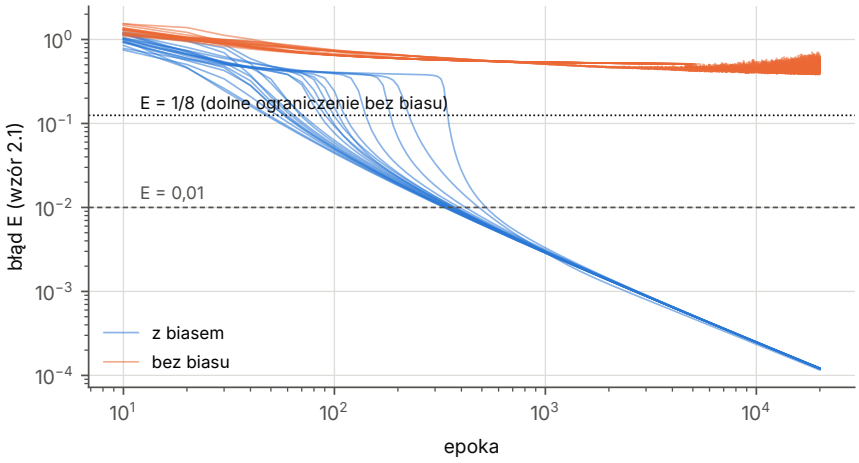
Algorytm 2.1 w materiałach jest opisany dla jednego wzorca. Przy czterech wzorcach powtarzamy go dla każdego wzorca po kolei (w losowej kolejności w każdej epoce).

4.4 Wynik z biasem: jak czytać Tabelę 1

wzorzec (wejście)	y_1	y_2	y_3	y_4
1	**0,998**	0,006	0,005	0,000
2	0,004	**0,997**	0,000	0,004
3	0,006	0,000	**0,998**	0,007
4	0,000	0,005	0,004	**0,997**

Wiersz to wzorzec podany na wejście, a **kolumna** to odpowiedź jednego neuronu wyjściowego. Idealny wynik to macierz jednostkowa (jedynki na przekątnej, zera poza nią). Na przekątnej mamy co najmniej 0,997, a poza nią najwyżej 0,007, więc sieć kopiuje poprawnie. Dokładnych 0 i 1 nie zobaczymy nigdy, bo sigmoida osiąga je dopiero „w nieskończoności”.

Wszystkie 20 treningów z różnych startów się udało. Błąd spadł poniżej 0,01 po około 360 epokach.



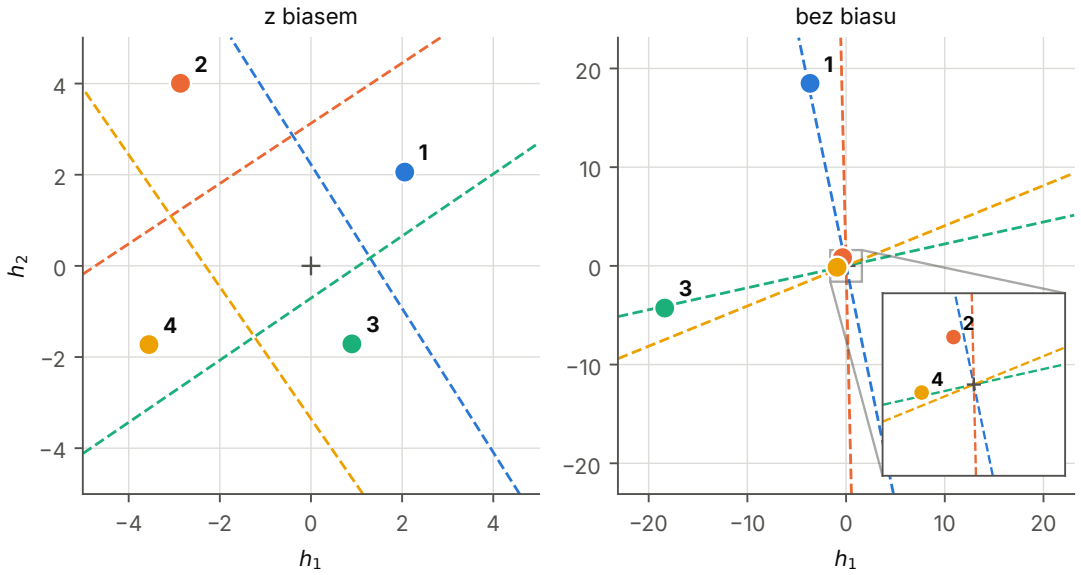
Rysunek 5: Błąd E dla 20 treningów z biasem (niebieskie, spadają do zera) i 20 bez biasu (pomarańczowe, zatrzymują się wysoko).

4.5 Granica decyzyjna

Neuron wyjściowy daje $y > 0{,}5$, gdy $s > 0$ („to mój wzorzec”), a $y < 0{,}5$, gdy $s < 0$ („to nie mój”). **Granica decyzyjna** to zbiór punktów, w których $s = 0$:

$$w_1 h_1 + w_2 h_2 + b = 0,$$

gdzie b to waga biasu. To **równanie prostej** na płaszczyźnie (h_1, h_2), która dzieli płaszczyznę na stronę „tak” i stronę „nie”. Kopiowanie działa, gdy **prosta każdego neuronu odcina jego własny punkt od pozostałych trzech**.



Rysunek 6: Kody wzorców 1–4 w warstwie ukrytej (kropki) i granice decyzyjne neuronów wyjściowych (przerywane, w kolorze neuronu). Z lewej z biasem, z prawej bez biasu.

Na lewym panelu cztery wzorce leżą w rogach czworokąta, a każda prosta odcina „swój” róg. Na prawym (bez biasu) wszystkie proste przechodzą przez środek układu $(0, 0)$ i nie potrafią tego zrobić.

4.6 Bias dokładnie

Czym jest: dodatkowe wejście neuronu, które zawsze ma wartość 1. Jego waga b jest trenowana jak każda inna. Neuron liczy więc $\mathbf{w} \cdot \mathbf{h} + b$, czyli do sumy dochodzi stała, której sieć sama się uczy.

Geometrycznie: to wyraz wolny w prostej $y = ax + b$. Bez biasu granica $w_1 h_1 + w_2 h_2 = 0$ **musi przechodzić przez** $(0, 0)$, więc da się ją tylko obracać. Z biasem można ją dodatkowo **przesunąć** w dowolne miejsce, na odległość $|b|/\|\mathbf{w}\|$ od środka.

Neuron wyjściowy nr 1 z naszej sieci: $w_1 = 3{,}21$, $w_2 = 2{,}03$, **bias** $b = -4{,}53$

wzorzec	kod (h_1, h_2)	s bez biasu	y bez biasu	s z biasem	y z biasem	cel
1	(2,06; 2,06)	+10,79	1,000	+6,27	**0,998**	1
2	(−2,86; 4,01)	−1,05	**0,259**	−5,58	0,004	0
3	(0,90; −1,71)	−0,60	**0,355**	−5,12	0,006	0
4	(−3,55; −1,73)	−14,93	0,000	−19,45	0,000	0

Bias −4,53 przesuwa wszystkie sumy w dół. Wzorzec 1 ma tak silny sygnał, że nadal daje 0,998, a wzorce 2 i 3, które bez biasu dawały niepewne 0,26 i 0,36, spadają prawie do zera.

Bias to próg zadziałania

Każdy neuron wyjściowy ma „świecić” tylko dla 1 z 4 wzorców, więc domyślnie powinien być wyłączony. Ujemny bias ustawia go właśnie w pozycji „wyłączony”, a włączyć go może tylko silny sygnał jego własnego wzorca. Wszystkie cztery wyuczone wagi biasu wyszły ujemne (od −2,8 do −6,5), więc sieć sama to odkryła. Bez biasu neuron przy słabym sygnale ($s \approx 0$) daje zawsze $f(0) = 0{,}5$, czyli jest „niezdecydowany”.

4.7 Bez biasu: dlaczego się nie da

- Żaden z 20 treningów bez biasu się nie udał. Błąd zatrzymał się na poziomie 0,41–0,61.
- Inny, niezależny optymalizator (L-BFGS, 300 startów) też nie zszedł niżej niż 0,258. Problem leży więc w architekturze, a nie w algorytmie treningu.
- Ciekawostka: największe wyjście zawsze było na właściwym neuronie. Sieć „wie", który to wzorzec, ale nie potrafi odtworzyć wartości 0 i 1.

Dowód w prostych słowach, że zawsze $E \geq \frac{1}{8}$

Gdyby błąd był mniejszy niż $\frac{1}{8}$, każde z 16 wyjść musiałoby być po właściwej stronie 0,5. Każda z czterech prostych (przez środek układu!) musiałaby wtedy oddzielić „swój" punkt od trzech pozostałych, czyli te trzy musiałyby mieścić się w półpłaszczyźnie, w kącie mniejszym niż 180°. Ułóżmy cztery punkty wokół środka jak godziny na tarczy zegara. Między sąsiednimi punktami są cztery „luki" kątowe o sumie 360°. Warunek dla punktu k oznacza, że dwie luki przy nim mają razem ponad 180°. Dla wszystkich czterech punktów naraz daje to $2 \cdot 360° > 4 \cdot 180°$, czyli $720° > 720°$, co jest sprzecznością. Bez biasu tej sieci nie da się więc wytrenować dla żadnych wag.

4.8 Wpływ współczynnika uczenia

η	0,05	0,1	0,2	0,5	1	2	5
epoki do $E < 0{,}01$ (mediana)	3559	1779	889	356	179	94	55
udane treningi	20/20	20/20	20/20	20/20	20/20	20/20	**17/20**

Liczba epok jest odwrotnie proporcjonalna do η (około $180/\eta$): dwa razy większy krok daje dwa razy krótszy trening. Przy $\eta = 5$ kroki są za duże, „przeskakują" minimum i trening staje się zawodny.

4.9 Odpowiedzi na pytania z polecenia

1. **Czy sieć dałoby się wytrenować bez biasu?** Nie. Bez biasu każda granica decyzyjna musi przechodzić przez środek układu, a czterech punktów nie da się ułożyć tak, żeby każdy odciąć od pozostałych prostą przez środek. Błąd zawsze wynosi co najmniej $\frac{1}{8}$.
2. **Jaka jest rola biasu?** To regulowany próg zadziałania neuronu (wyraz wolny prostej). Pozwala przesunąć granicę decyzyjną w dowolne miejsce i ustawić „domyślną" odpowiedź neuronu na 0, tak żeby włączał się tylko dla swojego wzorca.

5 Zadanie 4: sieć SOM do kompresji obrazów

Polecenie w skrócie

Na podstawie Wykładu 3 zaimplementuj w Pythonie sieć samoorganizującą się (SOM) i użyj jej do kompresji obrazów w skali szarości (128×128: `zelda`, `cameraman`, `peppers`). Dla ramki 2×2 piksele dobierz parametry nauki (sugerowane: 20 epok, $q = 10$, $\eta = 0{,}2$). Następnie dla początkowej liczby neuronów $4, 8, 16, 32, 64, 128, 256$ wykonaj trening, kompresję i dekompresję i narysuj zależność stopnia kompresji CR od jakości obrazu PSNR. Opcjonalnie zbadaj wpływ rozmiaru ramki, współczynnika q i rozdzielczości obrazu.

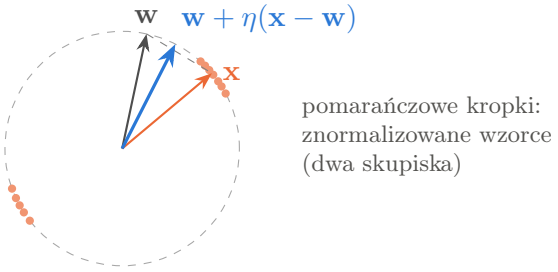
5.1 Co jest tu nowego: uczenie bez nauczyciela

W zadaniach 1 i 3 każdy wzorzec miał **wartość pożądaną** z, a trening zmniejszał różnicę między odpowiedzią sieci a z (uczenie *nadzorowane*). Tutaj wzorce to **same wejścia**, bez żadnych „poprawnych odpowiedzi" (uczenie *nienadzorowane*). Sieć ma sama znaleźć w danych **skupiska** podobnych wektorów i dla każdego wyznaczyć „typowego przedstawiciela".

Architektura jest taka sama jak MADALINE z zadania 2: warstwa neuronów liniowych, z których każdy liczy iloczyn skalarny (podobieństwo) znormalizowanego wejścia ze swoimi znormalizowanymi wagami. Różnica polega na tym, że w zadaniu 2 wagi wpisywaliśmy ręcznie (obrazy liter), a tu sieć sama ich szuka.

5.2 Jak uczy się sieć SOM (Algorytm 2.1)

1. Wagi wszystkich K neuronów losujemy z $[-1, 1]$ i normalizujemy do długości 1.
2. Dla każdego wzorca $\mathbf{x}$ (też znormalizowanego) liczymy wyjścia wszystkich neuronów i wybieramy **neuron zwycięski**, czyli ten z największym iloczynem skalarnym (najbardziej podobny do $\mathbf{x}$).
3. **Tylko zwycięzca** poprawia wagi: $\mathbf{w} \leftarrow \mathbf{w} + \eta(\mathbf{x} - \mathbf{w})$, a potem normalizuje je z powrotem do długości 1. Pozostałe neurony się nie zmieniają. To reguła **„zwycięzca bierze wszystko"** (ang. *winner takes all*, WTA).
4. Powtarzamy przez wiele epok.



Poprawka przesuwa wagi zwycięzcy kawałek w stronę wzorca. Po wielu epokach każdy neuron, który wygrywał, ustawia się w **środku „swojego" skupiska**. Neurony, które **nigdy nie wygrały**, nie zmieniają się wcale. To **neurony martwe**: nie reprezentują żadnych danych, więc po treningu się je usuwa.

5.3 Pomysł kompresji: słownik typowych ramek

1. Obraz 128×128 dzielimy na **ramki** 2×2 piksele, czyli 4096 ramek po 4 liczby.

2. Każdą ramkę rozkładamy na dwie części:
   - **długość** wektora $\|\mathbf{x}\|$, która mówi, jak jasna jest ramka,
   - **kierunek** $\mathbf{x}/\|\mathbf{x}\|$, który mówi, jaki ma „kształt” (np. jednolita, jaśniejsza z lewej, krawędź po przekątnej).
3. Sieć SOM uczy się na kierunkach wszystkich ramek. Wagi żywych neuronów to **słownik typowych kształtów**.
4. Zamiast 4 pikseli zapisujemy dla każdej ramki tylko dwie liczby całkowite: $\mathcal{L}$, czyli numer najbardziej podobnego neuronu, oraz $\mathcal{T} = \text{round}(\|\mathbf{x}\|/q)$, czyli jasność podzieloną przez współczynnik kwantyzacji q. Do pliku dochodzi jeszcze słownik $\mathcal{W}$ (wagi neuronów).
5. **Dekompresja:** ramka $= q \cdot \mathcal{T} \cdot \mathcal{W}[\mathcal{L}]$, czyli kształt ze słownika przeskalowany do zapamiętanej jasności.

Jedna ramka obrazu `zelda`, sieć z 64 neuronami, z których przeżyło 11

Ramka [100, 110, 120, 130]:

- długość $\sqrt{100^2 + 110^2 + 120^2 + 130^2} = 231{,}1$, więc $\mathcal{T} = \text{round}(231{,}1/10) = 23$,
- kierunek [0,433; 0,476; 0,519; 0,563]. Najbardziej podobny jest neuron nr 9 z wagami [0,490; 0,490; 0,482; 0,537] (kosinus 0,998), więc $\mathcal{L} = 9$,
- odtworzenie: $10 \cdot 23 \cdot [0{,}490;\ 0{,}490;\ 0{,}482;\ 0{,}537] \approx [113,\ 113,\ 111,\ 124]$.

Jasność ramki zgadza się, ale łagodny gradient $100 \to 130$ został „spłaszczony”, bo słownik z 11 kształtów nie ma dokładnie takiego. Na tym polega strata jakości. Więcej neuronów to bogatszy słownik i wierniejsze ramki.

5.4 Jak mierzymy wynik: CR i PSNR

PSNR (szczytowy stosunek sygnału do szumu) mierzy **jakość** obrazu odtworzonego:

$$\text{PSNR} = 10 \log_{10} \frac{255^2}{\text{MSE}} \ [\text{dB}],$$

gdzie MSE to średni kwadrat różnicy pikseli oryginału i obrazu odtworzonego. Większy PSNR oznacza lepszą jakość. Orientacyjnie: około 30 dB to dobra jakość z widocznymi drobnymi błędami, a powyżej 40 dB różnicy praktycznie nie widać.

CR (współczynnik kompresji) mówi, **ile razy mniejszy** jest plik skompresowany:

$$\text{CR} = \frac{\text{bity obrazu oryginalnego}}{\text{bity wag} + \text{bity indeksów } \mathcal{L} + \text{bity jasności } \mathcal{T}}.$$

Oryginał to 8 bitów na piksel, czyli 32 bity na ramkę 2×2. Indeks zajmuje tyle bitów, ile trzeba, żeby zapisać numer neuronu (np. 4 bity dla 11 neuronów). Jasność przy $q = 10$ mieści się w 6 bitach ($\mathcal{T} \le 51$). Wagi to 8 bitów na liczbę, ale przy ramce 2×2 to tylko 1–2% pliku.

Dlaczego dla ramki 2×2 CR nie przekroczy $\approx 4{,}6$

Nawet przy jednym neuronie każda ramka potrzebuje co najmniej $1 + 6 = 7$ bitów zamiast 32, więc $\text{CR} \le 32/7 \approx 4{,}57$. Każde podwojenie liczby żywych neuronów dokłada 1 bit na ramkę i obniża CR. Wyższą kompresję dają tylko większe ramki.

5.5 Program i eksperymenty

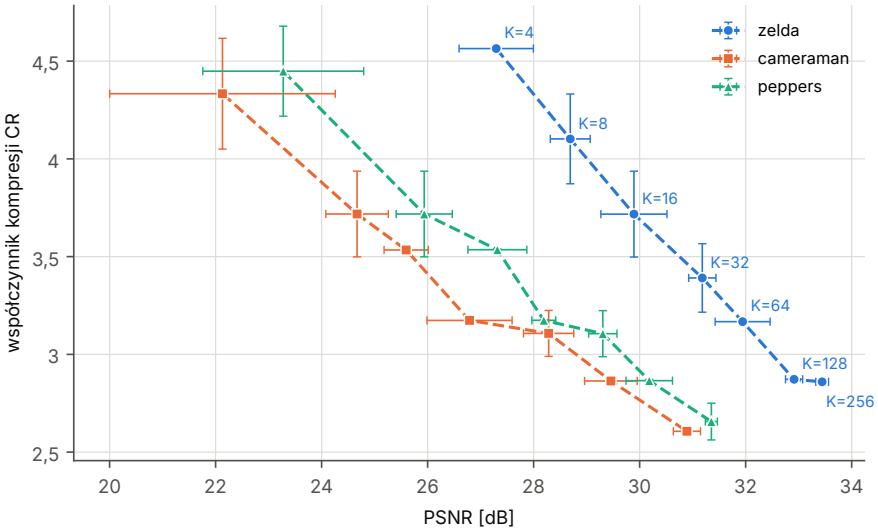
```
python som.py images/zelda.png --neurons 64 --out zelda_K64.png
```

Program wykonuje cały Algorytm 2.2 i wypisuje raport w formacie z wykładu: liczbę ramek i żywych neuronów, bity na indeks, długość i wagę, rozmiar oryginału i pliku skompresowanego, wagi neuronów, CR i PSNR. Opcjami można zmienić ramkę, liczbę epok, η, q i ziarno. Każdy wariant w eksperymentach uruchamialiśmy z 5 ziarnami i podajemy średnią.

Dobór parametrów (`zelda`, 64 neurony): η od 0,05 do 0,2 daje praktycznie ten sam PSNR (ok. 32 dB), a zbyt duże η (0,9) psuje wynik o ok. 2 dB. Po 5 epokach trening jest prawie zakończony. Większe q podnosi CR kosztem jakości ($q = 40$: CR 3,95, PSNR niższy o 2,5 dB). Zostaliśmy przy wartościach sugerowanych: 20 epok, $\eta = 0{,}2$, $q = 10$.

Główny wynik: wpływ liczby neuronów.

początkowe K	4	8	16	32	64	128	256
żywe neurony (`zelda`)	1,4	3,0	4,8	8,0	13,2	18,0	24,4
CR	4,56	4,10	3,72	3,39	3,17	2,87	2,86
PSNR [dB]	27,3	28,7	29,9	31,2	31,9	32,9	33,4



Rysunek 7: Charakterystyka CR–PSNR dla trzech obrazów: każdy punkt to inna początkowa liczba neuronów (od lewej $K = 4$ do $K = 256$).

Wykres pokazuje typowy **kompromis kompresji stratnej**: więcej neuronów daje lepszą jakość i słabszą kompresję. `cameraman` i `peppers` wypadają gorzej od `zelda` (o 2–5 dB), bo mają więcej ostrych krawędzi, których kilka typowych kształtów nie odda.

5.6 Najważniejsza obserwacja: neurony martwe

Z 256 neuronów przeżywa tylko ok. 24–35 (ok. 10%). Dlaczego?

Dane zajmują mały kawałek sfery

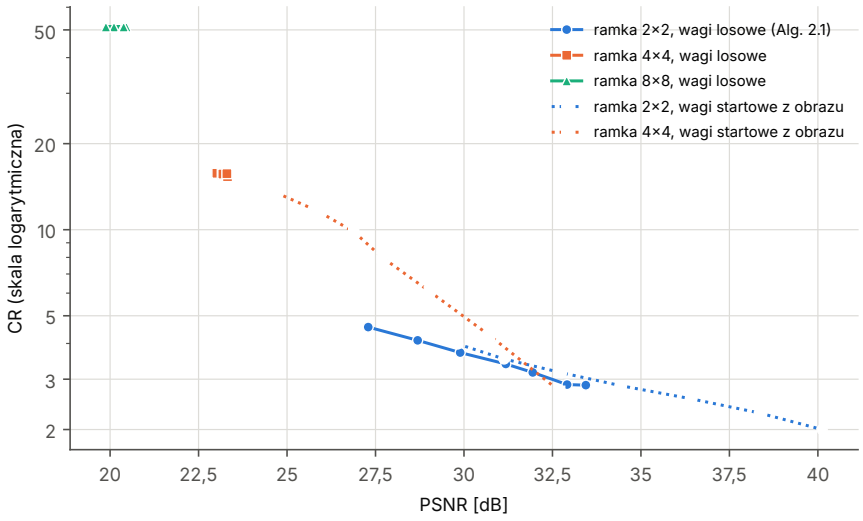
Piksele są nieujemne, a sąsiednie piksele mają podobne wartości. Kierunki wszystkich ramek leżą więc blisko kierunku „ramki jednolitej” $[\frac{1}{2}, \frac{1}{2}, \frac{1}{2}, \frac{1}{2}]$: w obrazie `zelda` połowa ramek ma z nim kosinus powyżej 0,997. Wagi losowane z $[-1, 1]$ są rozrzucone po całej sferze, a w ten mały obszar trafia tylko ok. 10% z nich. Reszta nigdy nie jest najbliżej żadnej ramki, więc nigdy



Rysunek 8: Obrazy oryginalne i odtworzone dla 4, 16 i 256 neuronów (ramka 2×2).

nie wygrywa i zostaje martwa. Dla ramek 4×4 (wektory 16-wymiarowe) w obszar danych trafia już tylko 0,1% losowych wektorów, a dla 8×8 praktycznie żaden. Wtedy przeżywa jeden neuron i sieć zapamiętuje tylko jasność ramek.

W dodatkowym eksperymencie wagi startowe ustawiliśmy na losowe ramki obrazu (zamiast liczb z $[-1, 1]$). Wtedy przeżywają wszystkie neurony, a dla ramki 2×2 i 256 neuronów PSNR rośnie z 33,4 do **40,1 dB**. Ramka 4×4 zaczyna dawać sensowne wyniki: od CR 13,5 przy 24,7 dB do CR 2,7 przy 32,7 dB.



Rysunek 9: CR–PSNR dla różnych ramek i sposobów inicjalizacji wag (`zelda`). Wagi losowe przy ramkach 4×4 i 8×8 dają jeden żywy neuron, więc punkty zlewają się w jedno miejsce.

5.7 Wnioski z zadania 4

- SOM uczy się bez wartości pożądanych: neurony same ustawiają się w środkach skupisk danych. W kompresji te środki tworzą słownik typowych kształtów ramek.
- Liczba neuronów steruje kompromisem: więcej neuronów to wyższy PSNR i niższe CR. Dla ramki 2×2 CR mieści się w zakresie ok. 2,6–4,6, a PSNR 22–33 dB.
- Sugerowane parametry (20 epok, $\eta = 0{,}2$, $q = 10$) są dobre. Wyniki słabo zależą od η i liczby epok, a q to dodatkowe pokrętło kompromisu.
- Główne ograniczenie to neurony martwe: przy losowych wagach z $[-1, 1]$ przeżywa ok. 10% neuronów dla ramki 2×2 i tylko jeden dla większych ramek. Inicjalizacja wag ramkami obrazu rozwiązuje ten problem.

6 Pytania kontrolne z odpowiedziami

Zadanie 1

1. **Dlaczego przy $N < M$ neuronu nie da się nauczyć bezbłędnie?**
   Bo M równań z $N < M$ niewiadomymi jest (dla losowych danych) sprzecznych: żadne wagi nie spełniają wszystkich naraz. Można tylko zminimalizować błąd.
2. **Dlaczego przy $N < M$ i $N = M$ wszystkie uruchomienia dają te same wagi?**
   Bo funkcja błędu ma jedno minimum (jedną „miskę"), a każdy start do niego prowadzi.
3. **Dlaczego przy $N > M$ wagi końcowe są różne, choć błąd jest zerowy?**
   Bo rozwiązań jest nieskończenie wiele, a reguła delta zmienia wagi tylko w kierunkach wzorców. Reszta wag zostaje taka, jak wylosowano na starcie.
4. **Skąd się biorą wagi początkowe?**
   Losujemy je z przedziału $(-1, 1)$, z innym ziarnem generatora w każdym uruchomieniu (100–104), żeby były różne, ale powtarzalne.
5. **Dlaczego wynik reguły delta dla $N < M$ nieco różni się od X^+z?**
   Bo wagi są poprawiane po każdym wzorcu i krążą wokół minimum. Odchyłka jest proporcjonalna do η i znika dla $\eta \to 0$.
6. **Co robi operator `@`?**
   Mnożenie macierzowe w NumPy: dla dwóch wektorów iloczyn skalarny (`w @ x`), dla macierzy i wektora odpowiedzi na wszystkie wzorce naraz (`X @ w`).

Zadanie 2

7. **Dlaczego wektory trzeba normalizować?**
   Bez normalizacji iloczyn skalarny zależałby od liczby czarnych pikseli, a nie tylko od kształtu. Po normalizacji jest kosinusem kąta, czyli miarą podobieństwa od 0 do 1.
8. **Dlaczego ta sieć nie wymaga treningu?**
   Bo wagi każdego neuronu to po prostu znormalizowany obraz jego litery. Nie trzeba ich szukać.
9. **Czy pewność 0,45 oznacza 45%?**
   Nie. To kosinus kąta między obrazem a wzorcem. O poprawności decyduje to, czy właściwy neuron ma większe wyjście niż pozostałe.
10. **Kiedy podobieństwo wynosi 1, a kiedy 0?**
    1 dla obrazu identycznego ze wzorcem. 0 dla obrazu bez ani jednego wspólnego czarnego piksela.
11. **Dlaczego przy dużym szumie wszystko jest rozpoznawane jako „m"?**
    Bo „m" ma najwięcej czarnych pikseli i „łapie" najwięcej losowych czarnych punktów szumu.
12. **Dlaczego przesunięcie o kilka pikseli szkodzi bardziej niż szum?**
    Bo sieć porównuje piksele na tych samych pozycjach. Przesunięta litera prawie nie pokrywa się ze wzorcem, a zaszumiona nadal leży w tym samym miejscu.

Zadanie 3

13. **Po co warstwa ukryta ma tylko 2 neurony?**
    Żeby zadanie było nietrywialne: sieć musi „ścisnąć" 4 wzorce do 2 liczb (wąskie gardło) i sama wymyślić kod.

14. **Co to jest granica decyzyjna?**
Prosta $w_1h_1 + w_2h_2 + b = 0$ w przestrzeni warstwy ukrytej. Po jednej stronie neuron daje $y > 0{,}5$, po drugiej $y < 0{,}5$.

15. **Czym jest bias i dlaczego bez niego sieć się nie uczy?**
Stałe wejście 1 z trenowaną wagą, czyli próg neuronu. Bez niego granice muszą przechodzić przez $(0, 0)$ i nie da się odciąć każdego z 4 wzorców od pozostałych.

16. **Dlaczego wyjścia nie są dokładnie 0 i 1?**
Bo sigmoida osiąga 0 i 1 dopiero w nieskończoności. Można się do nich tylko zbliżać.

17. **Skąd nazwa „propagacja wsteczna”?**
Błędy liczy się najpierw na wyjściu, a potem przesyła wstecz przez wagi do warstwy ukrytej.

18. **Jak η wpływa na trening?**
Większe η to szybszy trening (liczba epok $\approx 180/\eta$), ale zbyt duże (tu 5) psuje stabilność.

Zadanie 4

19. **Czym różni się uczenie nienadzorowane od nadzorowanego?**
W nadzorowanym każdy wzorzec ma wartość pożądaną i minimalizujemy błąd. W nienadzorowanym są tylko wejścia, a sieć sama szuka w nich skupisk.

20. **Co oznacza reguła „zwycięzca bierze wszystko”?**
Dla każdego wzorca wagi poprawia tylko neuron o największym wyjściu (najbardziej podobny): $\mathbf{w} \leftarrow \mathbf{w} + \eta(\mathbf{x} - \mathbf{w})$ i normalizacja.

21. **Co to jest neuron martwy i skąd ich tak wiele?**
Neuron, który nigdy nie wygrał, więc jego wagi się nie zmieniły. Ramki obrazu zajmują mały obszar sfery, a losowe wagi z $[-1, 1]$ trafiają tam rzadko (ok. 10% dla ramki 2×2).

22. **Co zapisujemy w pliku zamiast pikseli?**
Dla każdej ramki numer neuronu zwycięskiego ($\mathcal{L}$) i skwantowaną jasność ($\mathcal{T}$), a raz dla całego obrazu wagi neuronów ($\mathcal{W}$).

23. **Co mierzą CR i PSNR?**
CR mówi, ile razy plik jest mniejszy od oryginału. PSNR mierzy jakość obrazu odtworzonego w decybelach (im więcej, tym lepiej).

24. **Dlaczego więcej neuronów to lepsza jakość, ale gorsza kompresja?**
Bogatszy słownik kształtów wierniej oddaje ramki, ale numer neuronu wymaga więcej bitów.

25. **Do czego służy współczynnik q?**
Dzieli jasność ramki przed zaokrągleniem. Większe q to mniej bitów na jasność (wyższe CR), ale większy błąd jasności (niższy PSNR).

7 Słowniczek

Bias (neuron obciążający) stałe wejście 1 z trenowaną wagą. Działa jak próg neuronu i pozwala przesunąć granicę decyzyjną.

CR (współczynnik kompresji) ile razy plik skompresowany jest mniejszy od oryginału.

Epoka jedno przejście przez wszystkie wzorce treningowe.

Funkcja błędu (E, MSE) liczba mierząca, jak bardzo odpowiedzi sieci różnią się od wartości pożądanych. Trening ją minimalizuje.

Funkcja sigmoidalna $f(s) = 1/(1 + e^{-s})$, zamienia dowolną liczbę na wartość z $(0, 1)$.

Gradient kierunek najszybszego wzrostu funkcji błędu. Trening idzie w przeciwną stronę.

Granica decyzyjna zbiór wejść, dla których neuron jest dokładnie „na granicy" (dla sigmoidy $y = 0{,}5$). Dla neuronu z dwoma wejściami jest to prosta.

Iloczyn skalarny $\mathbf{w} \cdot \mathbf{x} = \sum w_i x_i$. Dla wektorów o długości 1 równy kosinusowi kąta, czyli miara podobieństwa.

Kwantyzacja wektorowa zastępowanie wektora numerem najbliższego wektora ze słownika. Tak działa kompresja SOM.

MADALINE jednowarstwowa sieć neuronów liniowych (*Multiple ADAptive LINear Elements*).

Neuron liniowy liczy iloczyn skalarny wag i wejść: $y = \mathbf{w} \cdot \mathbf{x}$.

Neuron martwy neuron sieci SOM, który ani razu nie wygrał, więc jego wagi się nie zmieniły. Usuwa się go po treningu.

Neuron zwycięski neuron z największym wyjściem. Jego litera jest wynikiem rozpoznania.

Normalizacja podzielenie wektora przez jego długość, co daje wektor o długości 1.

Perceptron wielowarstwowy (MLP) sieć z kilkoma warstwami neuronów, przez które sygnał płynie tylko w jedną stronę: od wejścia do wyjścia.

Propagacja wsteczna metoda treningu MLP: błędy z wyjścia przesyła się wstecz, żeby policzyć poprawki wag we wszystkich warstwach.

Pseudoodwrotność X^+ uogólnienie macierzy odwrotnej. $X^+\mathbf{z}$ daje najlepsze rozwiązanie układu $X\mathbf{w} = \mathbf{z}$ (najmniejszy błąd, a przy wielu rozwiązaniach najkrótszy wektor wag).

PSNR szczytowy stosunek sygnału do szumu w dB, czyli miara jakości obrazu odtworzonego.

Ramka kwadratowy fragment obrazu (tu 2×2 piksele) traktowany jako jeden wektor.

Reguła delta $\mathbf{w} \leftarrow \mathbf{w} + \eta(z - y)\mathbf{x}$, czyli poprawka wag neuronu liniowego po jednym wzorcu.

SOM sieć samoorganizująca się: warstwa neuronów liniowych uczona bez nauczyciela, których wagi ustawiają się w środkach skupisk danych.

Uczenie nienadzorowane trening bez wartości pożądanych. Sieć sama znajduje strukturę danych.

Wektoryzacja zamiana obrazu (tablicy 2D) na wektor przez połączenie wierszy jeden za drugim.

Współczynnik pewności (cnf) wyjście neuronu zwycięskiego w zadaniu 2, kosinus kąta między obrazem a wzorcem. Nie jest to prawdopodobieństwo.

Współczynnik uczenia η długość kroku przy poprawianiu wag.

WTA (zwycięzca bierze wszystko) reguła, w której wagi poprawia tylko neuron zwycięski.

Wzorzec treningowy para (wejście, wartość pożądana).

Ziarno (seed) liczba ustawiająca generator losowy, dzięki której „losowe" liczby są powtarzalne.

Dodatek: pliki i uruchamianie

Zadanie	Pliki	Uruchomienie
1	`neuron_liniowy.py` (trening, eksperymenty), `wykresy.py`	`python wykresy.py` (liczy wszystko i rysuje wykresy)
2	`font_generator.py`, `madaline_ocr.py`, `eksperymenty.py`, `wykresy.py`, `times.ttf`	`python madaline_ocr.py training_patterns test_patterns_30`; wszystkie testy: `python eksperymenty.py`
3	`mlp.py` (sieć i trening), `eksperymenty.py`, `wykresy.py`	`python mlp.py` (szybki test z biasem i bez); wszystko: `python eksperymenty.py`
4	`som.py` (SOM, kompresja, CR i PSNR), `eksperymenty.py`, `wykresy.py`, `images/`	`python som.py images/zelda.png --neurons 64`; wszystko: `python eksperymenty.py`

Wymagane biblioteki: `numpy`, `matplotlib`, w zadaniach 2 i 4 dodatkowo `pillow`, a w zadaniu 3 `scipy`.